

KOHLER AI Bathroom Designer & Planner

Main Development Prompts (Documentation)

1. Competition-Grade Project Foundation

Prompt used:

Build the KOHLER AI Bathroom Designer & Planner as a competition-grade working prototype based on the existing project specification.

Use a React 18 + Vite + TypeScript frontend and FastAPI/Python backend. Structure the project cleanly with frontend, backend, AI, engine, data, tests, and documentation components.

Implement real KOHLER product catalog/product schemas, bathroom dimensions and budget inputs, style and fixture requirements, grounded product retrieval, constraint-aware spatial planning, 2D floor-plan visualization, AI-assisted requirement understanding, design alternatives, sustainability analysis, BOM generation, and interactive 3D visualization.

Build a real end-to-end application rather than a static mockup. Keep the architecture modular, testable and extensible. Run backend tests and the frontend production build after implementation and fix errors before finishing.

2. Spatial Planning & 2D Layout Engine

Prompt used:

Implement the bathroom spatial-planning engine.

Use actual product dimensions and room dimensions to generate feasible layouts. Implement 2D geometry primitives, rectangle/rotation handling, collision detection, boundary containment, door-swing validation, fixture clearances, wet/dry zoning, spatial validation, and automatic layout generation.

Add APIs for layout generation and validation and integrate the result into the frontend 2D floor-plan view.

The layout must not place fixtures at arbitrary coordinates. It must respect room boundaries, fixture dimensions, clearances, door swing and practical bathroom circulation.

Add tests for geometry, collisions, boundaries, door swing and layout validation. Keep the implementation deterministic and make sure the frontend build passes.

3. Product Retrieval, RAG & Design Alternatives

Prompt used:

Implement grounded KOHLER product retrieval and design-alternative generation.

Use the existing KOHLER catalog as the source of truth. Support semantic/product retrieval using the project's retrieval approach and metadata filtering such as category, price, style, water efficiency and compatibility.

Return grounded product information and evidence rather than inventing products.

Build multi-objective scoring for spatial feasibility, budget, style, functionality, compatibility, sustainability and user priorities.

Generate materially different suites such as Balanced, Luxury, Eco, Space Saver and Personalized.

Integrate spatial validation into the optimizer so alternatives are feasible. Add APIs, frontend UI, tests, and ensure the production build passes.

4. AI Requirement Understanding, Vision & Copilot

Prompt used:

Add the AI intelligence layer to the bathroom designer.

Implement natural-language requirement extraction, multimodal image/floor-plan analysis, conversational design state, natural-language design modifications, grounded explanations, and automatic revalidation after modifications.

Understand room dimensions, budget, preferred style, fixture needs and constraints.

For image/floor-plan input, extract useful information such as room structure, approximate dimensions, color palette and plumbing-wall orientation when available.

The conversational copilot must modify the existing design rather than unnecessarily restarting. Preserve relevant fixtures and constraints during targeted changes, and re-run spatial validation after modifications.

5. Sustainability & What-If Analysis

Prompt used:

Implement a sustainability and decision-support engine.

For selected products/designs, calculate annual water consumption where relevant, compare against a baseline, estimate annual water/utility savings, and provide clearly labeled assumptions for carbon-related calculations.

Add a what-if engine for product substitutions, budget changes and efficiency changes.

Expose results through backend APIs and a clear frontend comparison view. Make assumptions transparent and never present estimates as exact measurements.

6. BOM / Procurement Output

Prompt used:

Implement a procurement-oriented Bill of Materials for the active bathroom design.

Use actual selected KOHLER products/SKUs from the catalog. Include product name, category, SKU/model, quantity, list price and available efficiency information.

Calculate subtotal, GST at the configured rate, clearly labeled rough-in/installation allowance, and grand total.

Support JSON, CSV, Markdown and printable output.

The BOM must always reflect the active design bundle and update when products are removed or the bundle changes.

7. Product-Specific 3D Visualization

Prompt used:

Make the 3D bathroom viewer product-specific.

The active design bundle must be the single source of truth for the 3D scene.

For each selected product, map the actual SKU to an appropriate visual profile and use product-specific geometry/material parameters where available. Make visibly different physical product designs look meaningfully different. Use verified dimensions where available and do not invent manufacturer specifications.

The 3D scene must update when the active bundle changes. Clearing the bundle must produce an empty bathroom shell with no leftover fixtures.

Preserve camera controls, finish switching, clearance visualization, water-flow effects and product inspection. Do not use one hardcoded generic fixture for all products in a category.

8. Spatial Arrangement Refinement — 2D + 3D

Prompt used:

Fix the spatial arrangement using the current implementation rather than rewriting the entire spatial engine.

Use the 2D layout as the single source of truth for both 2D and 3D.

Ensure the vanity is against a practical wall and away from the door swing; the toilet is against an appropriate wall with correct orientation; the shower is against a wall/corner; and the bathtub, when present, is along a practical wall.

Ensure wall-dependent fixtures are correctly oriented, fixtures do not overlap, door swing is unobstructed, clearances and circulation are respected, actual product dimensions are used, and awkward empty spaces are minimized.

Do not use random coordinates or separate placement logic for 3D. Generate/validate the layout once, then render that exact placement in both 2D and 3D.

Test multiple room sizes and fixture combinations so the solution is not hardcoded to one layout.

9. Shower Door + Shower Head Integration

Prompt used:

Fix the shower-door and shower-head integration.

The active bundle is the source of truth. A shower door and shower head are separate physical products and both must be represented when both are selected.

Ensure the selected shower-door SKU determines its visual representation, different shower-door products can have appropriate procedural geometry, the shower head remains visible when a shower door is present, and shower head, hand shower, trim/diverter and shower door can coexist.

Position shower components inside the shower area. Keep 2D and 3D representations consistent with the active bundle. Clearing or switching bundles must remove/replace the correct products.

Inspect category filtering, deduplication, rendering order, depth/occlusion and hardcoded shower geometry causing components to disappear.

10. Final End-to-End Integration

Prompt used:

Review the existing KOHLER AI Bathroom Designer end-to-end without restarting the project.

Verify the flow: user requirements → AI understanding → product retrieval → design alternatives → spatial validation → 2D layout → 3D visualization → sustainability/what-if → BOM.

Check that all major components use the same active design/bundle state and that changing or clearing a design propagates correctly across the application.

Fix integration issues, stale state, hardcoded demo data and inconsistent representations.

Run the full backend test suite and frontend production build. Fix failures before finishing. Do not remove working functionality just to make tests pass.

(These prompts capture the main development/refinement stages used for the working prototype. Minor debugging, URL verification, Git/GitHub, and repository-management prompts are intentionally omitted.)